



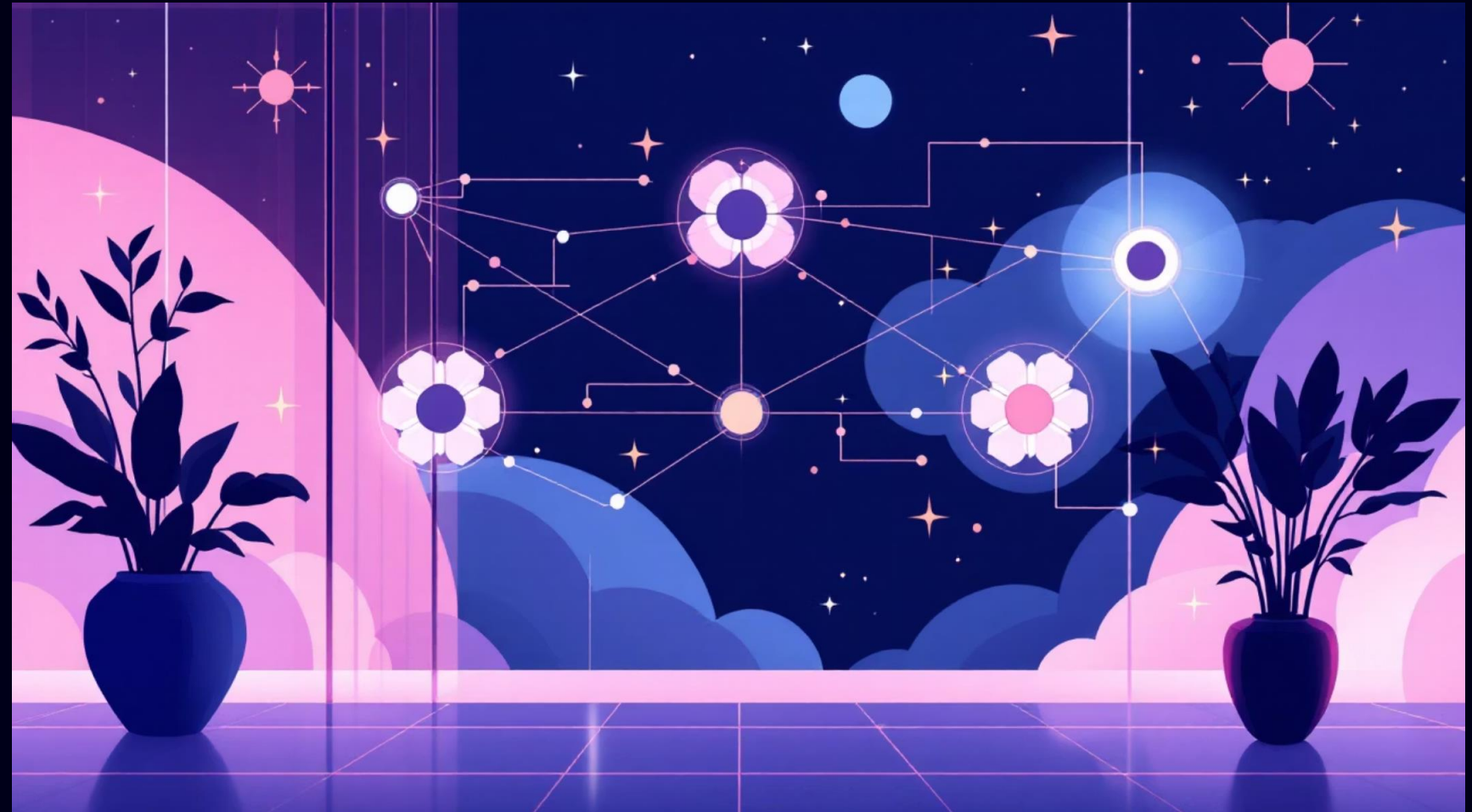
Image Dehazing Pipeline for Road Safety and Smart Cities

A complete transformer-based solution using DehazeFormer to improve visibility in hazy and smoggy conditions, enabling safer traffic surveillance and intelligent infrastructure systems.

Project Architecture Overview

Core Technology

DehazeFormer transformer architecture with window-based multi-head self-attention processes $3 \times 256 \times 256$ RGB inputs through 12 transformer blocks, extracting multi-scale features with skip connections for precise haze removal.



787,907 Parameters

12 transformer blocks with 8 attention heads and 64-dimensional embeddings

19 Pretrained Models

Across indoor, outdoor, RESIDE-6K, and RS-Haze datasets

8 Model Sizes

From tiny (2.9 MB) to large (98.2 MB) variants

Interactive Web Dashboard

Built with Streamlit, the dashboard provides three unified tabs for dehazing operations, synthetic haze generation, and automated benchmark evaluation.



01

Dehaze Image

Upload hazy images and select from 19 pretrained models to generate dehazed output with instant SSIM/PSNR metrics

02

Generate Haze

Apply synthetic haze to clear images using three methods: atmospheric scattering, Gaussian blur with color shift, or depth-based progressive haze

03

Dataset Evaluation

Run automated SSIM and PSNR calculations across I-HAZE, NH-HAZE, and Dense-Haze benchmarks with detailed log file generation

Evaluation Methodology

Benchmark Datasets

Three standard datasets provide comprehensive testing coverage:

- **I-HAZE**: 35 indoor synthetic haze pairs with train/validation/test splits
- **NH-HAZE**: 55 non-homogeneous real-world haze images
- **Dense-Haze**: 55 dense real-world haze scenarios

Per-image SSIM and PSNR values computed against ground truth, with dataset averages logged automatically.



Total Image Pairs



Datasets Tested

SSIM

Structural Similarity Index measures perceptual similarity (0 to 1, higher = better)

PSNR

Peak Signal-to-Noise Ratio measures pixel-level fidelity in decibels (typical range: 15–35 dB)



Applications in Road Safety



Traffic Surveillance

Real-time dehazing enables continuous monitoring of road conditions during fog, smog, and adverse weather events.



Smart City Infrastructure

Enhanced visibility for automated traffic management systems, license plate recognition, and incident detection.



Road Safety Systems

Improved input quality for autonomous vehicle perception systems and advanced driver assistance features.

Technical Implementation Details

Haze Generation Methods

Three synthetic approaches simulate realistic atmospheric conditions:

1. Atmospheric Scattering:

Physics-based model using $I(x) = J(x) \cdot e^{-\beta d} + A(1 - e^{-\beta d})$

2. Gaussian Blur + Color Shift: Fast approximation with white haze overlay

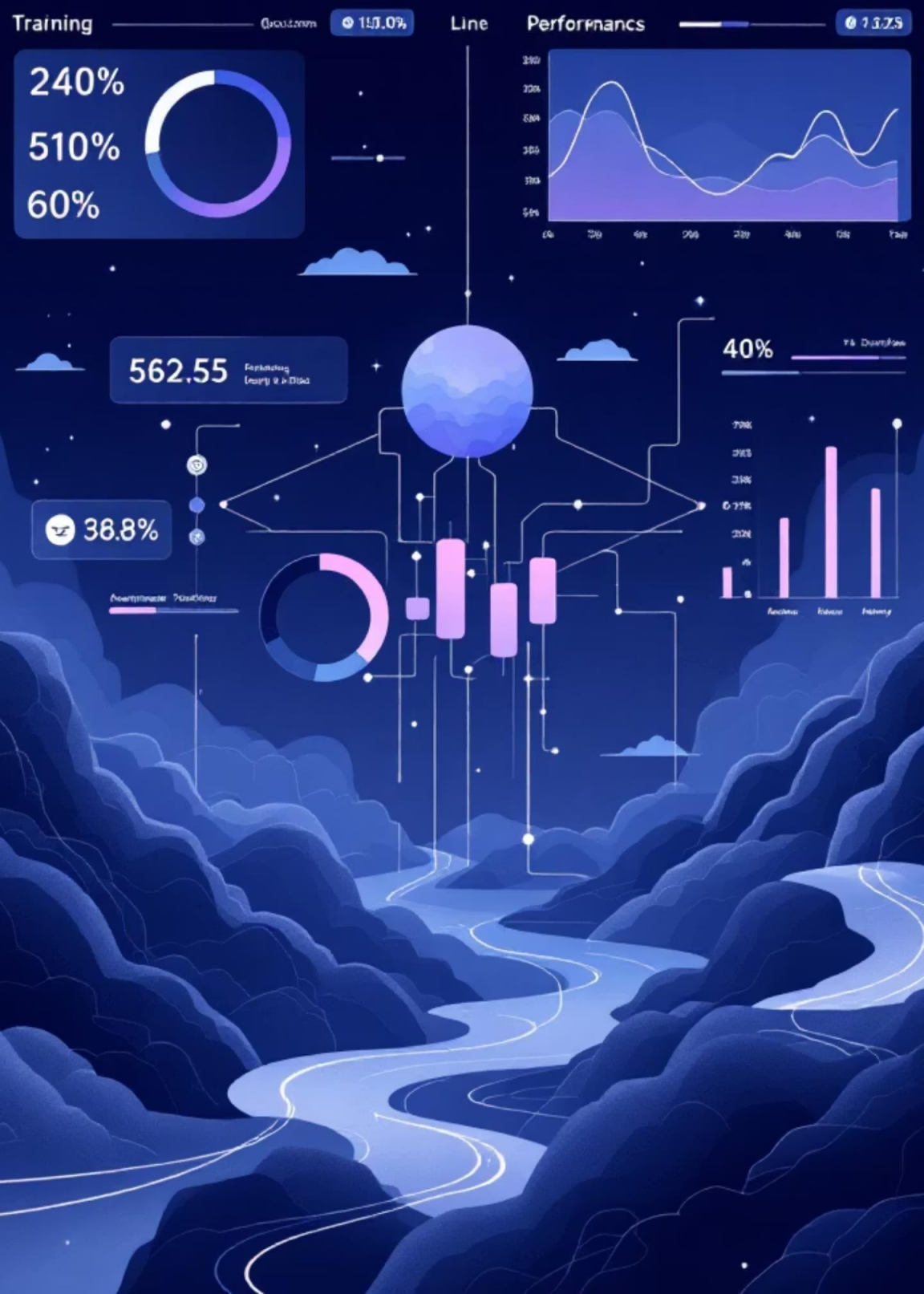
3. Depth-Based Progressive: More haze at image bottom simulating depth perception

Alternative Architectures

Beyond DehazeFormer, the codebase implements:

- **DehazeNet**: Lightweight CNN with residual blocks
- **AODNet**: All-in-one encoder-decoder with skip connections
- **PFFNet**: Progressive fusion framework
- **Dark Channel Prior**: Classical non-neural baseline





Model Performance Comparison

Architecture variants span multiple scales, from compact models suitable for edge deployment to high-capacity networks for server-based processing.



Transformer-Based

DehazeFormer uses window-based attention mechanisms for global context awareness



Multi-Scale Features

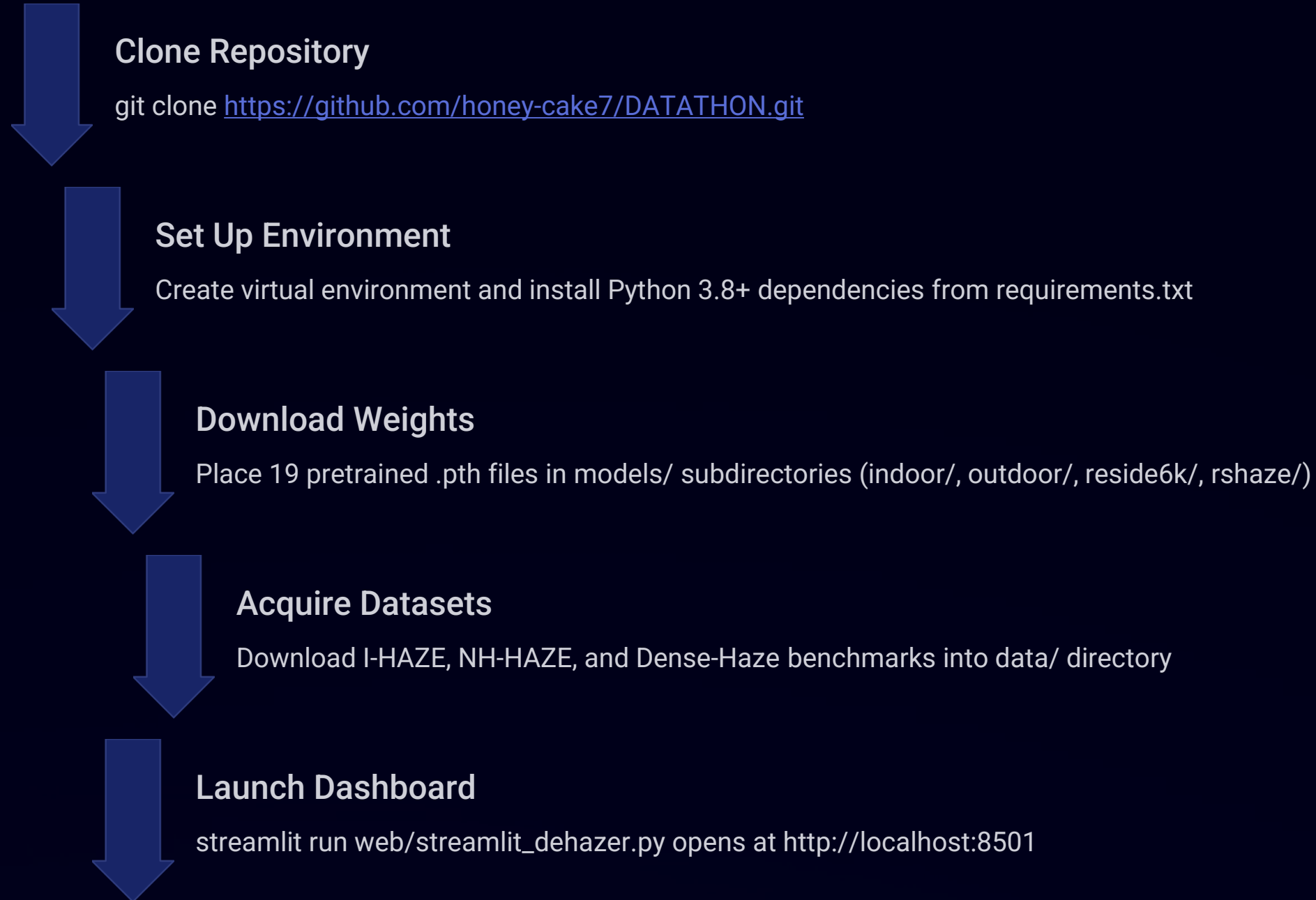
Extracts hierarchical representations with skip connections for detail preservation



Flexible Deployment

Tiny to large variants balance accuracy and computational requirements

Installation and Quick Start



Project Structure and Organization

```
├─ config.yaml
├─ requirements.txt
├─ README.md
├─ web/
│   └─ streamlit_dehazer.py
├─ src/
│   ├── models/
│   ├── inference/
│   ├── metrics/
│   └─ haze_generation/
├─ models/
├─ data/
├─ results/
├─ notebooks/
└─ scripts/
```

Key Components

- **src/models/architectures.py**: All neural network implementations
- **inference/dehaze.py**: Core dehazing pipeline
- **metrics/evaluation.py**: SSIM, PSNR, MSE, MAE calculation
- **haze_generation/generator.py**: Three synthetic methods
- **results/logs/**: Evaluation output with per-dataset .log files

Team and Acknowledgments

Aniket Patil

Sujith Pedapati

Lakshya Patidar

Vishwajeet Singh Bhati

This project demonstrates practical application of transformer architectures for real-world computer vision challenges in transportation and infrastructure systems.